Complete RetrievalLab Functionality Testing Guide

Setup — Make Sure Everything Is Running

bash
```
# Terminal 1 — Backend
cd ~/Documents/RetrievalLab
source .venv/bin/activate
.venv/bin/uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000

# Terminal 2 — Frontend
cd ~/Documents/RetrievalLab/frontend
npm run dev
```

Open two browser tabs: **http://localhost:3000** and **http://localhost:8000/docs**

PART 1 — Test via React UI (localhost:3000)

Dashboard (`/`)

What to check:

- 3 corpus cards visible in the table (legal_v1, finance_v1, healthcare_v1)
- All show `status: ready`
- NDCG trend chart renders with 3 colored lines
- Domain performance bars animate on load
- Quick Actions panel shows 4 buttons
- System Status bottom-left shows API Server, PostgreSQL, Vector Index all green

Expected: Everything loads within 2 seconds, metrics animate in, no blank cards

---

Corpora Page (`/corpus`)

What to check:

1. All 3 corpus cards appear with domain icons (🏥 📈 ⚖️)
2. Each card shows Docs: 1, Chunks: 1, strategy name
3. Click **Ingest Corpus** button → modal opens with all fields
4. Fill in:

    - Corpus ID: `test_corpus_v1`
    - Source: `data/seeds/healthcare/`
    - Domain: `healthcare`
    - Strategy: `recursive`
5. Click **Ingest** → should show success toast

Expected: Modal closes, new corpus appears in grid

Retrieval Playground (`/retrieve`)

Test 1 — Hybrid mode:

- Select corpus: `healthcare_v1`
- Query: `what is cardiovascular disease`
- Mode: **Hybrid**
- Click **Search**
- Expected: 1-2 results with text and score bars animating

Test 2 — Sparse mode:

- Same query, switch to **Sparse**
- Click **Search**
- Expected: results with BM25 scores (slightly different scores than hybrid)

Test 3 — Dense mode:

- Switch to **Dense**
- Expected: either results or "no results" (needs OpenAI embeddings)

Test 4 — Different corpus:

- Switch to `finance_v1`
- Query: `what is operating margin`
- Expected: finance content returned

Test 5 — Options panel:

- Click **Options**
- Drag Top K slider to 5
- Search again
- Expected: up to 5 results

AI Agent (`/agent`)

Requires ANTHROPIC_API_KEY in .env

Test 1 — Basic agent query:

- Select corpus: `healthcare_v1`
- Query: `What are the diagnostic criteria for type 2 diabetes?`
- Mode: Hybrid
- Click **Run Agent**
- Watch the 5 pipeline nodes animate one by one
- Expected: synthesized answer + sources list + confidence badge + agent trace

Test 2 — Expand sources:

- Click any source item to expand
- Expected: full chunk text revealed

Test 3 — Agent trace:

- Click **Agent Trace** at bottom
- Expected: dropdown shows 5+ trace lines with node names and latencies

Test 4 — Finance query:

- Switch to `finance_v1`
- Query: `What was the revenue growth rate?`
- Expected: finance-domain answer with source attribution

Eval Engine (`/eval`)

Test 1 — Basic scoring:

- Retrieved IDs box: paste exactly this

```
chunk_a
chunk_b
chunk_c
chunk_d
chunk_e
```

- Relevant IDs box:

```
chunk_a: 1.0
chunk_c: 2.0
```

- Query: `test medical query`
- Click **Compute Metrics**
- Expected: NDCG@10 ~0.85, MRR: 1.0, radar chart appears, all metrics listed

Test 2 — No relevant in top results:

- Retrieved IDs: `chunk_x`, `chunk_y`, `chunk_z`
- Relevant IDs: `chunk_a: 1.0`
- Expected: NDCG@10 = 0, MRR = 0, Hit Rate = 0

Test 3 — Perfect retrieval:

- Retrieved IDs: `chunk_a`, `chunk_b`
- Relevant IDs: `chunk_a: 1.0`, `chunk_b: 1.0`
- Expected: NDCG@10 = 1.0, MRR = 1.0, Precision = 1.0

Adversarial Page (`/adversarial`)

- 6 attack cards visible: Typo Noise, Synonym Substitution, Irrelevant Injection, Query Truncation, Semantic Trap, Domain Shift
- Each card shows original vs attacked query example
- Robustness formula explained at bottom
- No interaction needed — this is documentation of your research

---

Metrics Page (`/metrics`)

- System status card shows overall health
- Component cards: PostgreSQL, Redis, MinIO, ChromaDB with latency badges
- Prometheus metrics table lists all 6 metric names
- Endpoint URLs shown at bottom

PART 2 — Test via Swagger UI (localhost:8000/docs)

Health Router

Test 1: `GET /api/v1/health/live`

- Click Try it out → Execute

- Expected: `{"status": "alive"}` — 200 OK

Test 2: `GET /api/v1/health`

- Execute
- Expected: JSON with `components` array showing postgresql, redis, minio — all healthy

---

Corpus Router

Test 1: `GET /api/v1/corpus/`

- Execute with no filters
- Expected: array of 3 corpora, all `status: ready`

Test 2: `GET /api/v1/corpus/{corpus_id}`

- corpus_id: `healthcare_v1`
- Execute
- Expected: full corpus object with fingerprint, chunk_count: 1, total_tokens: 77

Test 3: `GET /api/v1/corpus/{corpus_id}/chunks`

- corpus_id: `healthcare_v1`, limit: `5`
- Execute
- Expected: array with 1 chunk, full text visible

Test 4: `POST /api/v1/corpus/ingest`

- Body:

json
```
{
  "corpus_id": "demo_v1",
  "source": "data/seeds/healthcare/",
  "domain": "healthcare",
  "strategy": "recursive"
}
```

- Expected: `{"status": "queued", "message": "..."}`

Retrieve Router

Test 1 — Hybrid:

json
```
{
  "query": "what causes high blood pressure",
  "corpus_id": "healthcare_v1",
  "mode": "hybrid",
  "top_k": 5
}
```

Expected: `total_results: 2`, results with scores

Test 2 — Sparse only:

json
```
{
  "query": "cardiovascular disease",
  "corpus_id": "healthcare_v1",
  "mode": "sparse",
  "top_k": 3
}
```

Expected: BM25 results with scores around 0.01-0.05

Test 3 — Finance corpus:

json
```
{
  "query": "operating margin revenue",
  "corpus_id": "finance_v1",
  "mode": "hybrid",
  "top_k": 3
}
```

Expected: finance content returned

Eval Router

Test 1: `POST /api/v1/eval/score`

json
```
{
  "retrieved_ids": ["chunk_a", "chunk_b", "chunk_c", "chunk_d", "chunk_e"],
  "relevant_ids": {"chunk_a": 1.0, "chunk_c": 2.0},
  "query": "diabetes diagnosis"
}
```

Expected: all metrics computed — NDCG@10, MRR, MAP, Precision, Recall, Hit Rate

Test 2: `GET /api/v1/eval/metrics`
Expected: list of all supported metric names

Agent Router

Test: `POST /api/v1/agent/query`

json
```
{
  "query_text": "What is hypertension and how is it diagnosed?",
  "corpus_id": "healthcare_v1",
  "mode": "hybrid",
  "top_k": 5,
  "synthesize": true
}
```

Expected: full response with `answer`, `sources`, `confidence`, `trace`, `query_type`, `detected_domain`

Also test: `GET /api/v1/agent/status`
Expected: JSON listing all 5 pipeline nodes with status: ready

---

PART 3 — Terminal Verification

bash
```
# Full system check — run all at once
echo "=== Health ===" && curl -s http://localhost:8000/api/v1/health/live

echo "=== Corpora ===" && curl -s http://localhost:8000/api/v1/corpus/ | python3 -m json.tool | grep -E "corpus_id|status|chunk_count"

echo "=== Retrieve ===" && curl -s -X POST http://localhost:8000/api/v1/retrieve/ \
  -H "Content-Type: application/json" \
  -d '{"query":"cardiovascular disease","corpus_id":"healthcare_v1","mode":"hybrid","top_k":3}' \
  | python3 -m json.tool | grep -E "total_results|score"

echo "=== Eval ===" && curl -s -X POST http://localhost:8000/api/v1/eval/score \
  -H "Content-Type: application/json" \
  -d '{"retrieved_ids":["a","b","c"],"relevant_ids":{"a":1.0},"query":"test"}' \
  | python3 -m json.tool | grep -E "ndcg|mrr|map"
```

Summary Checklist

Feature	Test Location	Pass Condition

Health check	Swagger + UI Metrics page	200 alive
Corpus list	UI Dashboard + Swagger	3 corpora ready
Corpus detail	Swagger	Returns fingerprint + tokens
Chunk browser	Swagger	Returns actual text
Hybrid retrieval	UI Playground + Swagger	Results with scores
Sparse retrieval	UI Playground	BM25 scores returned
Eval scoring	UI Eval + Swagger	NDCG/MRR computed
Agent pipeline	UI Agent + Swagger	Answer + trace returned
Ingest new corpus	UI Corpora modal	Success toast
Frontend routing	UI all 7 pages	No blank screens